

DSA STUDY BLUEPRINT SERIES

19. Merge Sorted Arrays Next Permutation

Merging Sorted Arrays and Finding Lexicographical Next Permutation

Section 03 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Merge Sorted:** Combine two arrays in-place or with auxiliary memory.
- **Next Permutation:** Pivot search from end, swap with next greater, and reverse suffix.

Memory & Execution Blueprint

Next Permutation Steps for [1, 2, 5, 4, 3]:

1. Find Pivot Breakpoint: 2 (at index 1) • 2. Swap 2 with next greater 3 $\Rightarrow$ [1, 3, 5, 4, 2] • 3. Reverse suffix $\Rightarrow$ **[1, 3, 2, 4, 5]**

C++ Implementation Reference

Standard code layout and syntax

```
void nextPermutation(vector& nums) {  
    int i = nums.size() - 2;  
    while (i >= 0 && nums[i] >= nums[i+1]) i--;  
    if (i >= 0) {  
        int j = nums.size() - 1;  
        while (nums[j] <= nums[i]) j--;  
        swap(nums[i], nums[j]);  
    }  
    reverse(nums.begin() + i + 1, nums.end());  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Next Permutation	$O(N)$ time, $O(1)$ space
Merge Arrays	$O(N + M)$

Key Practices & Gotchas

- **Important:** C++ standard library provides this direct utility as `next_permutation()`.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run the Dutch National Flag 3-pointer partition step for `[2, 0, 1]` :

Iteration	Pointers [low, mid, high]	Value at mid	Swap & Pointer Adjustments	Array State
1	[0, 0, 2]	nums[0] = 2	Swap nums[mid] & nums[high]; high--	[1, 0, 2]
2	[0, 0, 1]	nums[0] = 1	nums[mid] is 1; mid++	[1, 0, 2]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Three-way Partitions:** Partitions arrays into three zones (less, equal, greater) in a single pass.
- **Next Permutation:** Find peak breakpoint, swap elements with next-largest suffix value, and reverse suffix inline.
- **Related LeetCode Problems:** #75 Sort Colors, #31 Next Permutation.